Max-Planck-Gymnasium · Informatik · Leistungskurs (erhöhtes Anforderungsniveau)

Nachschreibklausur

Die Kletterhalle

Datenstrukturen, Algorithmen und Rekursion – im Format der Abiturprüfung, Block 1

Name _____ Kurs _____ Datum _____

Bearbeitungszeit	180 Minuten einschließlich Einlesezeit
Bewertungseinheiten	60 BE (Anforderungsbereiche I / II / III: 18 / 32 / 10 BE)
Hilfsmittel	die als Hilfsmittel zugelassenen Abschnitte der „Ergänzenden Hinweise zum Kerncurriculum Informatik“ (Neufassung 2025, u. a. Zeichenketten-Operationen, Operationen von DynArray, Stack, Queue und BinTree, Struktogramm-Notation); kein Taschenrechner, kein Computer
Material	Abbildungen 1 bis 12 am Ende der Aufgabenstellung; die Tabellen in Abbildung 8 und Abbildung 11 füllen Sie direkt aus

Hinweise

- Bearbeiten Sie **alle** Aufgabenteile a) bis m). Schreiben Sie Ihre Lösungen auf das ausgeteilte Klausurpapier und geben Sie jeweils den Aufgabenteil an. Geben Sie das Material mit Ihrem Namen versehen zusammen mit Ihrer Bearbeitung ab.
- Implementierungen schreiben Sie in **Java** mit den Klassen der Ergänzenden Hinweise (DynArray, Stack, Queue, BinTree). Diese Klassen sind nicht generisch: Entnommene Inhalte werden gecastet, z. B. `Route r = (Route) tourenbuch.pop();`.
- Mögliche **Laufzeitfehler** (Entnehmen aus leerer Struktur, ungültiger Index, Zugriff auf leeren Baum) müssen durch geeignete Abfragen explizit ausgeschlossen werden.
- Für Zeichenketten sind nur die Operationen der Ergänzenden Hinweise zulässig; Zeichenketten werden mit equals verglichen.
- Struktogramme notieren Sie sprachunabhängig nach den Vorgaben der Ergänzenden Hinweise (Zuweisung mit ←, Rückgabe mit „gib … zurück“).
- Die Bewertungseinheiten (BE) stehen bei jedem Aufgabenteil. Planen Sie etwa 2,5 Minuten je BE ein.

Bewertung (wird von der Lehrkraft ausgefüllt)

Teil	a)	b)	c)	d)	e)	f)	g)	h)	i)	j)	k)	l)	m)	Summe
max. BE	4	3	7	6	5	5	4	4	5	5	2	3	7	60
erreicht														

Notenpunkte: _____ Datum, Unterschrift: _____

Aufgabe

Die Kletterhalle **Vertikal** bietet Seilrouten unterschiedlicher Schwierigkeit, mehrere Toprope-Stationen und einen Boulderbereich. Ihre Mitglieder führen ein digitales Tourenbuch, einmal im Monat findet ein Hallenwettkampf statt, und die besten Mitglieder stehen in einer Bestenliste. Die Software der Halle ist als Klassendiagramm in Abbildung 1 im Material dargestellt.

a) Erläutern Sie am Beispiel der Klasse Mitglied die Funktion der beiden Konstruktoren sowie die Bedeutung der Kennzeichnungen + und – im Klassendiagramm. **[4 BE]**

Neben den Seilrouten bietet die Halle **Boulder** an: kurze Routen in Absprunghöhe, die ohne Seil geklettert werden. Ein Boulder hat wie jede Route einen Namen, einen Schwierigkeitsgrad und einen Sektor. Zusätzlich besitzt er eine Anzahl von Zonengriffen, für die es im Wettkampf Teilpunkte gibt. Boulder sollen ebenfalls in der dynamischen Reihung routen der Klasse Kletterhalle verwaltet werden.

b) Ergänzen Sie das Klassendiagramm aus Abbildung 1 um eine Klasse Boulder mit dem zusätzlichen Attribut, einem Konstruktor und einer Operation, die die Anzahl der Zonengriffe zurückgibt. Begründen Sie die gewählte Beziehung zwischen den Klassen Boulder und Route. **[3 BE]**

Jedes Mitglied führt ein digitales Tourenbuch. Es ist als Stapel tourenbuch vom Inhaltstyp Route modelliert: Die Operation klettern legt jede gekletterte Route oben auf den Stapel, die zuletzt gekletterte Route liegt also oben. Abbildung 2 zeigt das Tourenbuch des Mitglieds Mila.

c) Die Operation **hoechsterGrad(k: Ganzzahl): Ganzzahl** der Klasse Mitglied liefert den höchsten Schwierigkeitsgrad unter den k zuletzt gekletterten Routen. Enthält das Tourenbuch weniger als k Routen, werden alle Routen betrachtet. Ist das Tourenbuch leer oder ist k ≤ 0, wird 0 zurückgegeben. Nach Ausführung der Operation muss das Tourenbuch unverändert sein.

Beispiel: Für Mila (Abbildung 2) liefert hoechsterGrad(2) den Wert 6 und hoechsterGrad(10) den Wert 10.

Implementieren Sie die Operation hoechsterGrad in einer im Unterricht verwendeten Programmiersprache. **[7 BE]**

An jeder Toprope-Station hängt ein fest eingehängtes Seil. Wer dort klettern möchte, stellt sich in die Warteliste der Station, eine Schlange vom Inhaltstyp Mitglied (Abbildung 3). Wer das Warten aufgibt, meldet sich ab.

d) Die Operation **abmelden(name: Zeichenkette): Wahrheitswert** der Klasse Station entfernt das vorderste Mitglied mit dem übergebenen Namen aus der Warteliste. Die Reihenfolge der übrigen Mitglieder bleibt erhalten. Die Operation liefert wahr, wenn ein Mitglied entfernt wurde, sonst falsch.

Beispiel: Für die Warteliste aus Abbildung 3 liefert abmelden("Ada") den Wert wahr; danach enthält die Warteliste von vorn Ben, Cem, Ada, Dana.

Entwickeln Sie die Operation abmelden als Struktogramm. **[6 BE]**

Die Routen der Halle werden in der dynamischen Reihung routen vom Inhaltstyp Route verwaltet (Abbildung 4). Alle paar Wochen schrauben die Routenbauer einen ganzen Sektor um: Alle Routen dieses Sektors werden abgebaut und später durch neue ersetzt.

e) Die Operation **ausschrauben(sektor: Zeichenkette): Ganzzahl** der Klasse Kletterhalle entfernt alle Routen des übergebenen Sektors aus routen und liefert ihre Anzahl. Die Reihenfolge der übrigen Routen bleibt erhalten.

Beispiel: Für die Routen aus Abbildung 4 liefert ausschrauben("Dach") den Wert 4; danach enthält routen nur noch die Routen Morgenrot, Sandmann und Wolkenkratzer.

Implementieren Sie die Operation ausschrauben in einer im Unterricht verwendeten Programmiersprache.

[5 BE]

---

f) Die Operation **schwersteRoute(sektor: Zeichenkette): Route** der Klasse Kletterhalle soll die Route mit dem höchsten Schwierigkeitsgrad im übergebenen Sektor liefern, bei Gleichstand die erste in routen. Gibt es im Sektor keine Route, soll null geliefert werden. Abbildung 5 zeigt eine Implementierung, die mehrere Fehler enthält. Beim Aufruf schwersteRoute("Kante") für die Routen aus Abbildung 4 kommt es zu einem Laufzeitfehler.

Analysieren Sie den Ablauf dieses Aufrufs. Erläutern Sie die Fehler der Implementierung und korrigieren Sie die Operation.

[5 BE]

Im Boulderbereich hängt eine Trainingswand mit Griffen in einem Raster. Sie wird in der zweidimensionalen Reihung wand vom Inhaltstyp Ganzzahl gespeichert (Abbildung 6): wand[z][s] gibt die Qualität des Griffs in Zeile z und Spalte s an (1 = sehr kleiner Griff bis 9 = sehr großer Griff, 0 = kein Griff). Zeile 0 ist die oberste Reihe.

Eine Trainings-App schlägt Anfängerinnen und Anfängern einen Weg nach oben vor: Von einem Griff aus wird jeweils der beste der höchstens drei Griffe schräg links darüber, direkt darüber und schräg rechts darüber gewählt. Die rekursive Operation **aufstieg(z: Ganzzahl, s: Ganzzahl): Ganzzahl** (Abbildung 7) liefert die Summe der Griffqualitäten entlang dieses Weges.

g) Dokumentieren Sie den Ablauf des Aufrufs aufstieg(4, 2) für die Wand aus Abbildung 6 in der Tabelle in Abbildung 8.

[4 BE]

---

h) Erläutern Sie das Prinzip der Rekursion anhand der Operation aufstieg.

Begründen Sie, dass die Rekursionstiefe bei einem Erstaufruf aufstieg(z, s) höchstens z beträgt und genau dann gleich z ist, wenn der vorgeschlagene Weg die oberste Reihe erreicht.

[4 BE]

Beim monatlichen Hallenwettkampf erhalten die Teilnehmenden Wettkampfpunkte. Für die Siegerehrung werden sie in einer statischen Reihung vom Inhaltstyp Mitglied mit der Operation **sortiere** (Abbildung 10) nach ihren Punkten geordnet. Abbildung 9 zeigt die Ergebnisse eines Wettkampfs in der Reihenfolge der Anmeldung.

i) Dokumentieren Sie den Ablauf der Operation sortiere für die Reihung aus Abbildung 9 in der Tabelle in Abbildung 11. Ein Eintrag erfolgt jeweils an der mit „Protokollierung“ gekennzeichneten Stelle.

[5 BE]

---

j) Erläutern Sie die Funktionsweise der Operation sortiere.

Bestimmen Sie die Anzahl der Vergleiche von Punktzahlen, die sortiere für die Reihung aus Abbildung 9 durchführt, und schätzen Sie ab, wie sich diese Anzahl verändert, wenn sich die Anzahl der Teilnehmenden verdoppelt.

[5 BE]

k) Untersuchen Sie anhand Ihrer Dokumentation aus Aufgabenteil i), ob die Operation sortiere die ursprüngliche Reihenfolge von Teilnehmenden mit gleicher Punktzahl beibehält.

[2 BE]

Die Bestenliste der Halle wird als binärer Suchbaum bestenliste vom Inhaltstyp Mitglied verwaltet, geordnet nach den Punkten der Mitglieder: Im linken Teilbaum eines Knotens stehen nur Mitglieder mit weniger Punkten, im rechten Teilbaum nur Mitglieder mit mehr Punkten. Alle Punktzahlen sind verschieden. Abbildung 12 zeigt die aktuelle Bestenliste.

l) Zeichnen Sie den Baum aus Abbildung 12, nachdem nacheinander Pia mit 50 Punkten und Uwe mit 77 Punkten eingefügt wurden.

Geben Sie an, in welcher Reihenfolge der Baum durchlaufen werden muss, damit die Mitglieder nach Punkten absteigend ausgegeben werden.

[3 BE]

Ein Mitglied möchte wissen, wen es mit seiner aktuellen Punktzahl als Nächstes in der Bestenliste überholen kann.

m) Die Operation **naechstesZiel(b: BinTree vom Inhaltstyp Mitglied, punkte: Ganzzahl): Mitglied** der Klasse Kletterhalle liefert aus dem Suchbaum b das Mitglied mit der kleinsten Punktzahl, die größer als punkte ist. Gibt es kein solches Mitglied, wird null geliefert. Die Operation arbeitet rekursiv und nutzt die Ordnung des Suchbaums: In jedem Schritt wird höchstens ein Teilbaum weiter durchsucht.

Beispiel: Für die Bestenliste aus Abbildung 12 liefert naechstesZiel(bestenliste, 50) das Mitglied Sam (55 Punkte) und naechstesZiel(bestenliste, 95) den Wert null.

Implementieren Sie die Operation naechstesZiel in einer im Unterricht verwendeten Programmiersprache.

[7 BE]

Summe: 60 BE

Material

zu den Aufgabenteilen a) bis m)

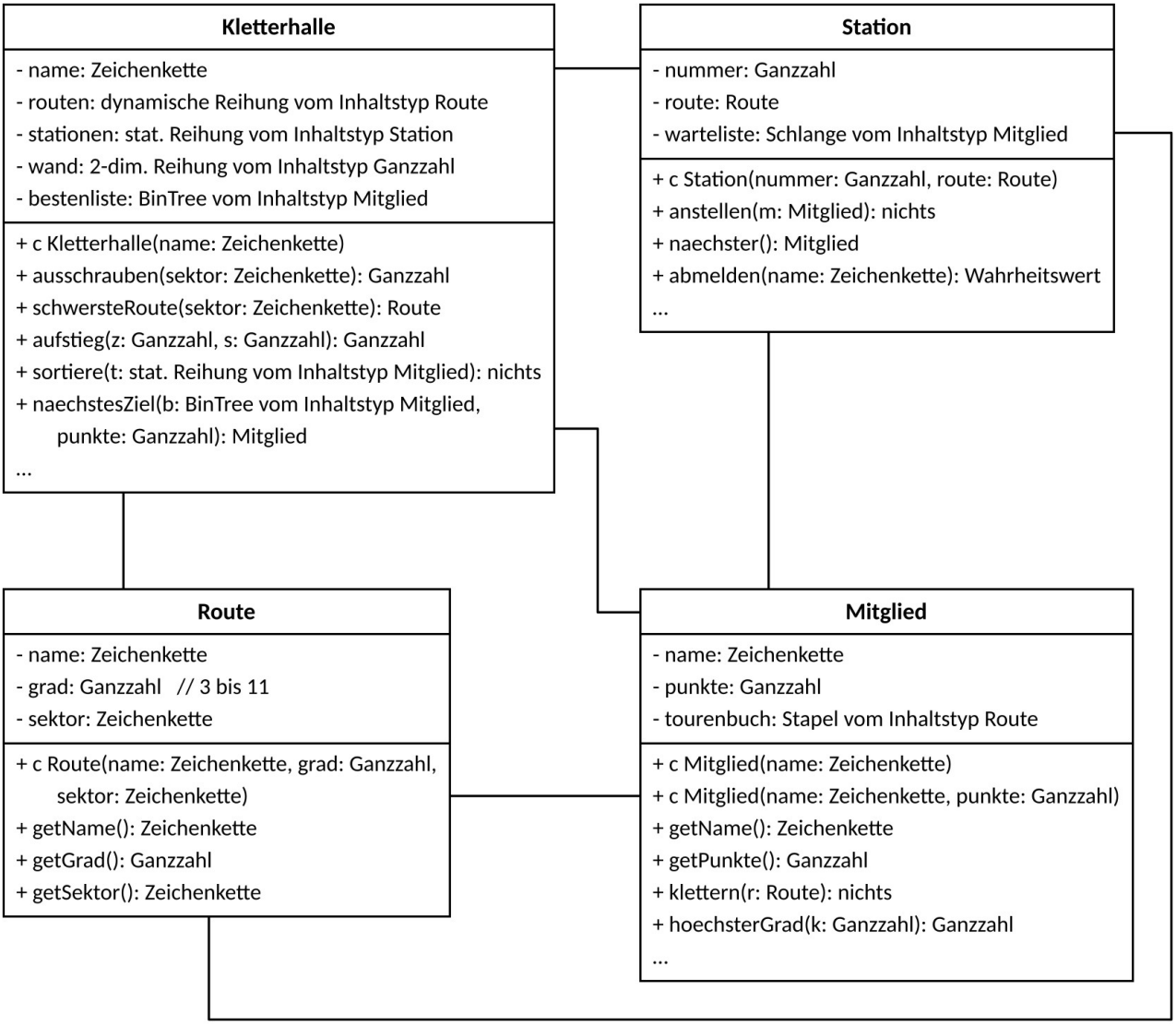


Abbildung 1: Klassendiagramm der Software der Kletterhalle (Ausschnitt)

zu Aufgabenteil c)

tourenbuch von Mila (Stapel)

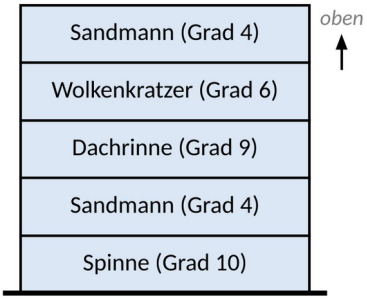


Abbildung 2: Tourenbuch des Mitglieds Mila (oben: zuletzt geklettert); in Klammern der Schwierigkeitsgrad

zu Aufgabenteil d)

warteliste der Station 2 (Schlange)

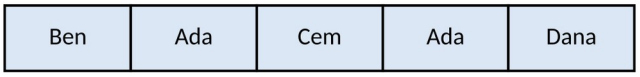


Abbildung 3: Warteliste der Station 2 (Mitglieder mit ihren Namen)

zu den Aufgabenteilen e) und f)

Index	0	1	2	3	4	5	6
Name	Morgenrot	Kalkfresser	Sandmann	Schwalbennest	Dachrinne	Wolkenkratzer	Spinne
grad	8	7	4	6	9	6	10
sektor	Platte	Dach	Platte	Dach	Dach	Kante	Dach

Abbildung 4: Inhalt der dynamischen Reihung routen (Name, Schwierigkeitsgrad und Sektor jeder Route)

```
1   public Route schwersteRoute(String sektor) {
2       Route beste = (Route) routen.getItem(0);
3       for (int i = 1; i <= routen.getLength(); i++) {
4           Route r = (Route) routen.getItem(i);
5           if (r.getSektor() == sektor && r.getGrad() > beste.getGrad()) {
6               beste = r;
7           }
8       }
9       return beste;
10  }
```

Abbildung 5: Fehlerhafte Implementierung der Operation schwersteRoute der Klasse Kletterhalle

zu den Aufgabenteilen g) und h)

z \ s	0	1	2	3	4
0 (oben)	3	0	5	5	0
1	0	4	0	6	1
2	2	0	7	0	3
3	5	3	0	2	0
4 (unten)	1	0	4	0	2

Abbildung 6: Inhalt der Reihung wand (Zeile 0 oben; 0 = kein Griff)

zu den Aufgabenteilen g) und h)

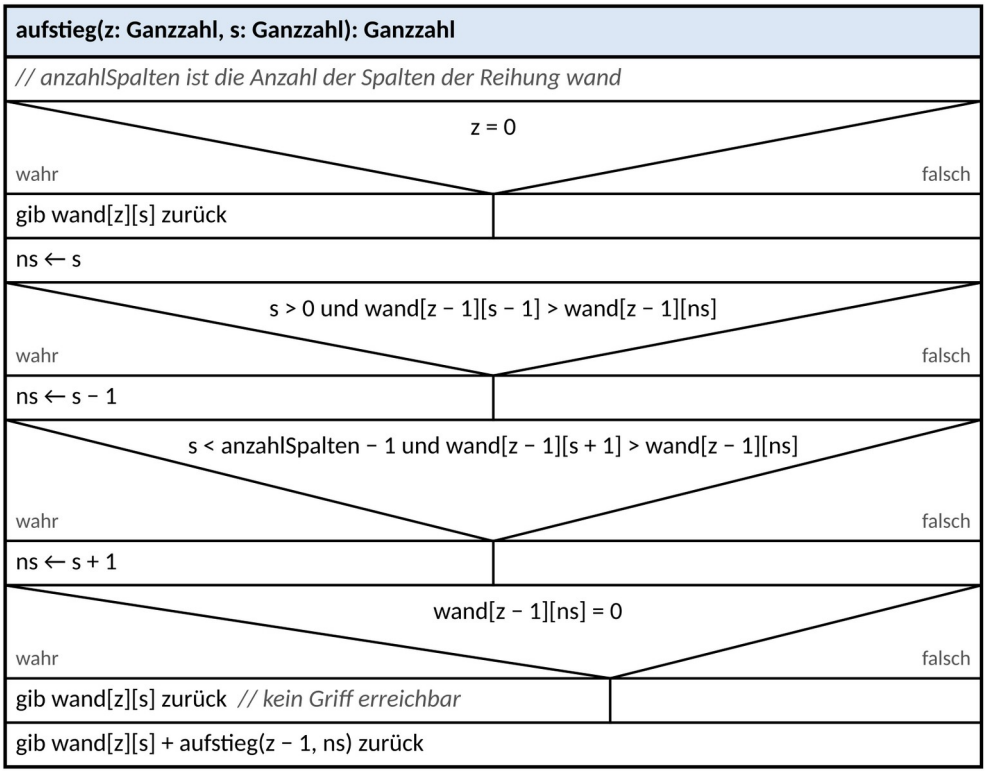


Abbildung 7: Struktogramm der Operation aufstieg der Klasse Kletterhalle

Rekursionstiefe	z	s	wand[z][s]	ns	Rückgabewert
0 (Erstaufruf)	4	2			

Abbildung 8: Tabelle zur Dokumentation des Aufrufs aufstieg(4, 2). Die erste Zeile gehört zum Erstaufruf. Bei jedem rekursiven Aufruf erhöht sich die Rekursionstiefe um 1. Endet ein Aufruf, bevor ns berechnet wird, tragen Sie in der Spalte ns „–“ ein. Nicht alle Zeilen müssen benötigt werden.

zu den Aufgabenteilen i) bis k)

Index	0	1	2	3	4	5
Name	Jana	Kai	Lea	Mo	Nele	Ole
Punkte	38	57	47	31	38	52

Abbildung 9: Reihung t der Teilnehmenden eines Wettkampfs mit ihren Wettkampfpunkten (Reihenfolge der Anmeldung)

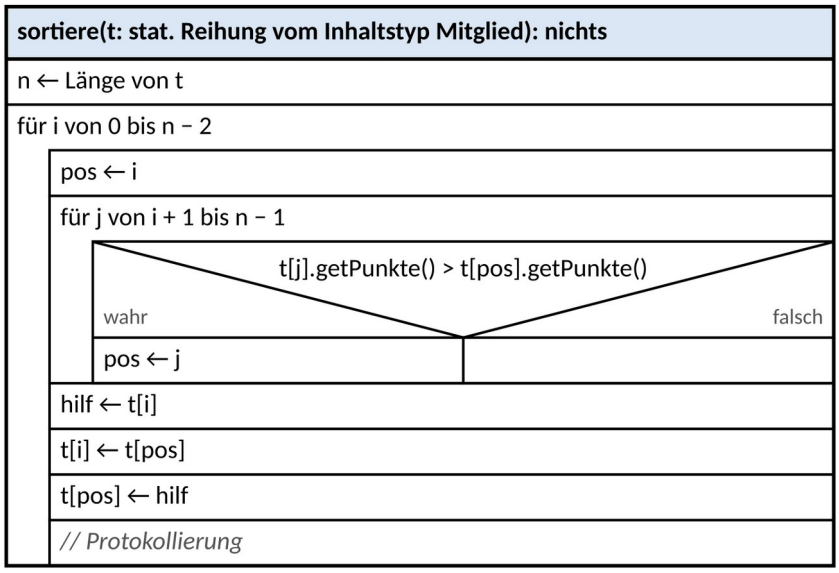


Abbildung 10: Struktogramm der Operation sortiere der Klasse Kletterhalle

i	pos	t[0]	t[1]	t[2]	t[3]	t[4]	t[5]
Start	–	Jana 38	Kai 57	Lea 47	Mo 31	Nele 38	Ole 52

Abbildung 11: Tracetabelle zur Vervollständigung. Tragen Sie an der Stelle „Protokollierung“ die Werte von i und pos sowie den Inhalt der Reihung t ein (Name und Punkte, z. B. „Kai 57“).

zu den Aufgabenteilen l) und m)

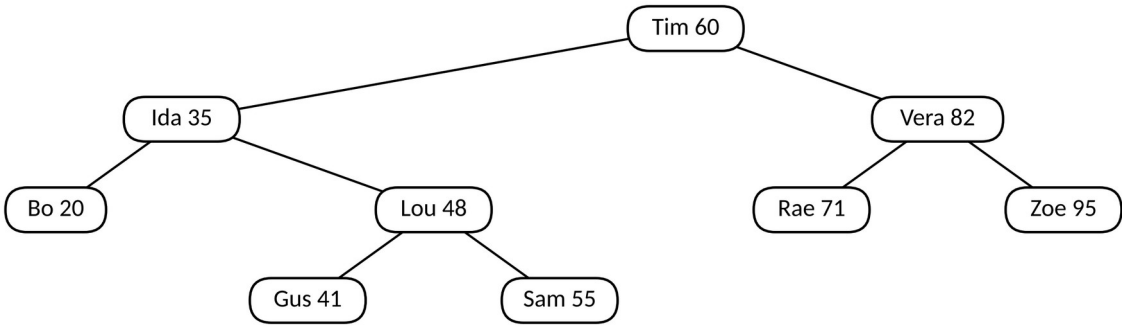


Abbildung 12: Binärer Suchbaum bestenliste (Name und Punkte; geordnet nach Punkten)